

Hostel Student Management System

A Data Structures mini project in C++ using **Arrays**, **Stacks** and **Strings**

Project	DS Project 2
Language	C++ (standard library only, no external libraries)
Interface	Console menu + web page (HTML & CSS) on localhost
Data structures	Array, Stack, String
Lines of code	approx. 420 (all hand written)

1. Objective of the Project

The aim of this project is to build a small record management system for a hostel and, while doing so, **implement the basic data structures by hand** instead of using ready made containers from the C++ standard library.

The system stores student records (roll number, name, course, room number) and supports:

- **Add** a new student
- **Display** all students
- **Search** a student by name, course or roll number
- **Delete** a student
- **Undo** the last deletion (this is the part that needs a stack)
- **Sort** the records alphabetically by name

The "undo delete" feature is the reason a stack is genuinely required. Deletions must be undone in **reverse order** of when they happened, which is exactly Last In First Out (LIFO) behaviour, so a stack is the natural structure for it.

2. Data Structures Used and Why

2.1 Array — to store the records

```
Student students[MAX];    // MAX = 100
int count_ = 0;           // how many students are actually stored
```

A single fixed size array holds every student record. `count_` tracks how many slots are in use, so the "logical size" of the list is independent of the array's physical capacity.

- **Insertion** puts the new record at index `count_` and increments it — $O(1)$.
- **Searching** walks the array from index 0 to `count_-1`, a **linear search** — $O(n)$.

- **Deletion** finds the position, then **shifts every later element one place left** so no empty hole is left in the middle — $O(n)$.

Why an array? Records need to be accessed by position and displayed in order, and the maximum number of students in a hostel is known in advance. An array gives constant time access with the simplest possible memory layout.

2.2 Stack — to remember deleted records (Undo)

```
Student stack_[MAX];    // the stack itself
int top_ = -1;          // index of the top element; -1 means empty
```

The stack is implemented using an array plus a `top_` index, which is the standard array implementation of a stack. It is used as an **undo history**: every time a student is deleted from the main array, that record is **pushed** onto the stack before it is removed. Choosing *Undo* **pops** the topmost record and re-inserts it into the array.

Operation	What the code does	Time
<code>push(s)</code>	increment <code>top_</code> , store the record at that index	$O(1)$
<code>pop()</code>	read the record at <code>top_</code> , then decrement <code>top_</code>	$O(1)$
<code>stackEmpty()</code>	returns true when <code>top_ == -1</code>	$O(1)$

Both the **overflow** condition (`top_ == MAX-1`, stack full) and the **underflow** condition (`top_ == -1`, nothing to pop) are checked before the operation, so the program can never write or read outside the array.

2.3 String — for names, courses and searching

`std::string` is used for the name and course fields, and string operations provide two of the features:

- **Case insensitive search:** `toLowerCase()` converts a string to small letters by adding 32 to the ASCII code of every capital letter; `contains()` then uses `find()` to check whether one string occurs inside another. This is why typing `bca` also finds `BCA`.
- **Sorting by name:** `sortByName()` is a **bubble sort** that compares two strings with the `>` operator, which compares them lexicographically (dictionary order), character by character.

3. Project Structure

```
ds_project_2/
|
+-- src/
|   +-- hostel.h      <-- ALL data structure code (array, stack, string)
|   +-- console.cpp   <-- menu driven console program
|   +-- server.cpp    <-- small web server that produces the web page
|
+-- web/
|   +-- index.html    <-- the user interface (plain HTML, no JavaScript)
|   +-- style.css     <-- the styling
|
+-- docs/report.html  <-- source of this document
+-- Makefile          <-- build commands
+-- README.md         <-- short instructions
+-- .gitignore
```

An important design point: **every data structure lives in one header file, `src/hostel.h`**. The console program and the web server contain no data structure logic of their own — they only provide two different ways of calling the same functions. This shows separation of logic from interface, and it means the same header can be reused inside a larger project later.

File	Contains	Approx. lines
<code>src/hostel.h</code>	Struct <code>Student</code> , the array, the stack, and all operations	140
<code>src/console.cpp</code>	<code>main()</code> with the text menu and <code>cin / cout</code>	100
<code>src/server.cpp</code>	Socket web server, form parsing, HTML generation	180
<code>web/index.html</code>	Forms, table and stack list, with placeholders	60
<code>web/style.css</code>	Colours, spacing, table and button styling	120

4. How to Compile and Run

Requirements: a C++ compiler (`g++`) and Linux or macOS. No libraries need to be installed. The web server uses POSIX sockets, which are built into the operating system.

4.1 The web version (recommended for the demo)

```
$ cd ds_project_2
$ make web

g++ src/server.cpp -o server
./server
Server running -> http://localhost:8080
```

Now open **http://localhost:8080** in any browser. Press **Ctrl + C** in the terminal to stop the server.

Important: run the server from inside the `ds_project_2` folder. It reads `web/index.html` and `web/style.css` using relative paths, so starting it from another folder gives a blank page.

4.2 The console version

```
$ cd ds_project_2
$ make cli

g++ src/console.cpp -o hostel
./hostel

===== HOSTEL STUDENT MANAGEMENT =====
1. Add student
2. Show all students
3. Search student
4. Delete student
5. Undo delete (stack)
6. Sort by name
7. Exit
Enter choice: _
```

4.3 Compiling by hand (without `make`)

```
$ g++ src/console.cpp -o hostel    && ./hostel
$ g++ src/server.cpp -o server    && ./server
```

Other commands: `make` builds both programs, `make clean` deletes the compiled files.

5. Code Walkthrough

These are the four functions worth showing to a teacher, because each one demonstrates a different data structure operation.

5.1 Insertion into the array

```
bool addStudent(int roll, string name, string course, int room) {
    if (count_ >= MAX)          return false;    // array is full
    if (findStudent(roll) != -1) return false;    // duplicate roll number

    students[count_].roll  = roll;
    students[count_].name  = name;
    students[count_].course = course;
    students[count_].room  = room;
    count_++;
    return true;
}
```

Two validity checks are done first (capacity and duplicate roll number), then the record is written at the end of the array. No shifting is needed, so this is $O(1)$.

5.2 Linear search in the array

```
int findStudent(int roll) {
    for (int i = 0; i < count_; i++)
        if (students[i].roll == roll)
            return i;          // found: return the position
    return -1;                 // not found
}
```

The function returns the **index**, not the record, because deletion needs to know *where* in the array the student is. Returning **-1** for "not found" is the usual convention.

5.3 Deletion: array shift plus stack push

```
bool deleteStudent(int roll) {
    int i = findStudent(roll);
    if (i == -1) return false;          // nothing to delete

    push(students[i]);                 // STACK: save it for undo

    for (int j = i; j < count_ - 1; j++) // ARRAY: shift everything left
        students[j] = students[j + 1];
    count_--;
    return true;
}
```

This is the most important function in the project because **both** structures work together. The order matters: the record is pushed onto the stack **before** it is overwritten by the shift, otherwise the data would be lost and undo would be impossible.

Deleting roll 102 from the array

```
before:  [101][102][103][104]    count_ = 4
          |
          +--- push a copy onto the stack
shift:   [101][103][104][104]    copy each element one place left
after:   [101][103][104]         count_ = 3   (last slot is ignored)

stack:   | 102 | <-- top_
```

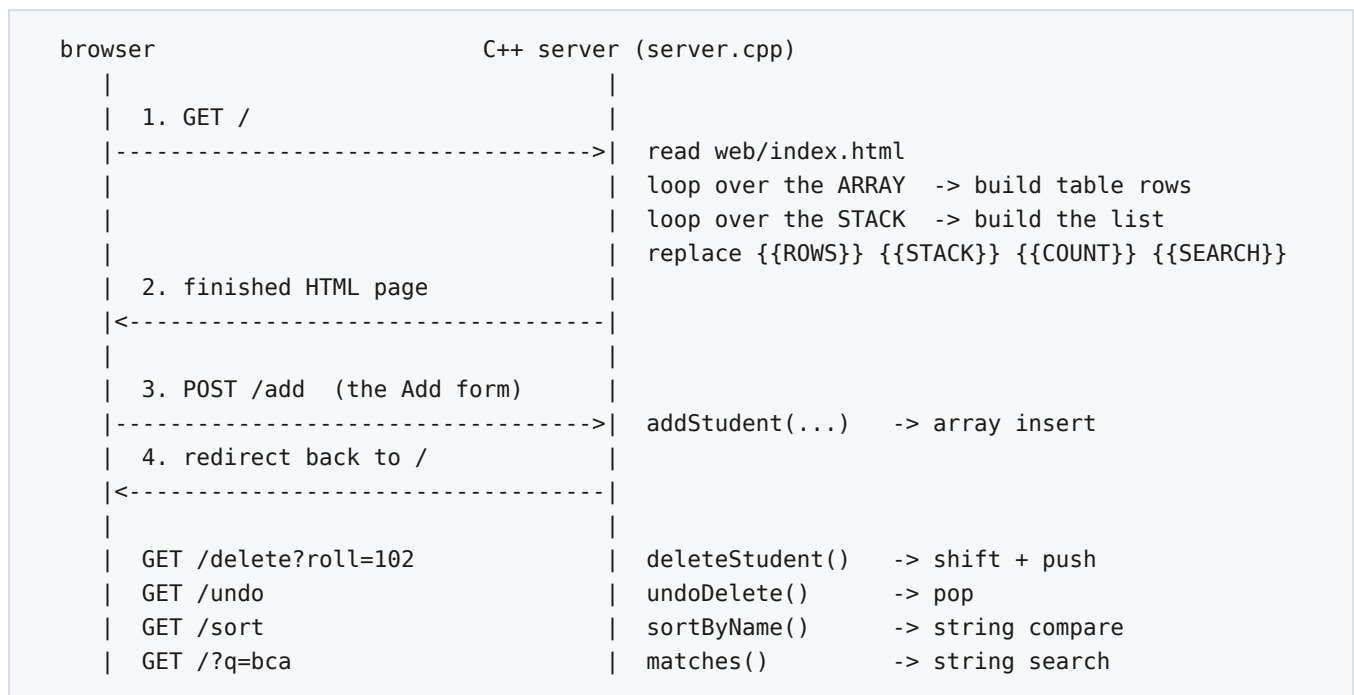
5.4 Undo: pop from the stack

```
bool undoDelete() {
    if (stackEmpty()) return false;    // nothing was deleted
    Student s = pop();                 // STACK: take the top record
    return addStudent(s.roll, s.name, s.course, s.room); // put it back
}
```

Undo simply pops the most recent deletion and re-inserts it using the normal insertion function. Because the stack is LIFO, deleting three students and pressing undo three times restores them in the **reverse** order of deletion, which is exactly how undo should behave.

6. How the Web Interface Works

The user interface is written in **HTML and CSS only** — **there is no JavaScript anywhere in this project**. Every button is an ordinary HTML form or link, and all the work is done in C++.



The server is a normal socket program and follows the classic five steps:

1. `socket()` — create the socket
2. `bind()` — attach it to port 8080
3. `listen()` — start accepting connections
4. `accept()` — wait for a browser, in an infinite loop
5. `read()` the request, send the reply with `write()`, then `close()`

The page is produced by **template replacement**: `web/index.html` contains the markers `{{ROWS}}`, `{{STACK}}`, `{{COUNT}}` and `{{SEARCH}}`, and the C++ function `makePage()` replaces each marker with text built from the array and the stack. This keeps design (HTML/CSS) completely separate from logic (C++).

A useful detail for the demo: the page displays **the stack itself** as a list with the top element labelled `TOP →`. So when a student is deleted, the record can be seen appearing on top of the stack, and when Undo is pressed it can be seen being removed. The data structure is visible on screen rather than hidden inside the program.

7. Testing

Every feature was tested in both versions. These are the actual results.

#	Test	Expected	Result
1	Add a student with a new roll number	Added, array size increases by 1	Pass
2	Add a student with a roll number that already exists	Rejected, array unchanged	Pass
3	Search <code>bca</code> (small letters)	Finds the student whose course is <code>BCA</code>	Pass
4	Search a roll number as text	Finds that student	Pass
5	Delete a student from the middle	Removed, later records shift left, no gap	Pass
6	Check the stack after deleting	Deleted record shown on top of the stack	Pass
7	Press Undo	Student returns to the array, stack becomes empty	Pass
8	Press Undo with an empty stack	Handled safely, no crash (underflow check)	Pass
9	Delete two students, then undo twice	Restored in reverse order (LIFO)	Pass
10	Sort by name	Anita, Karan, Priya, Ravi — alphabetical	Pass
11	Delete a roll number that does not exist	Reports not found, nothing pushed	Pass
12	Add a name containing a space through the web form	Stored correctly as "Priya Singh"	Pass

Time complexity summary

Operation	Structure used	Best	Worst	Reason
Add student	Array	$O(1)$	$O(n)$	insert is $O(1)$, but the duplicate check is a search
Search	Array + String	$O(1)$	$O(n)$	linear search, whole array in the worst case
Delete student	Array + Stack	$O(1)$	$O(n)$	shifting the elements after the deleted one
Push / Pop	Stack	$O(1)$	$O(1)$	only the <code>top_</code> index changes
Undo delete	Stack + Array	$O(1)$	$O(n)$	$O(1)$ pop, then a re-insert that checks duplicates
Sort by name	Array + String	$O(n)$	$O(n^2)$	bubble sort with nested loops

Space complexity is **$O(n)$** : one array of 100 records plus one stack of 100 records, allocated once.

8. Limitations and Future Scope

Current limitation	How it could be improved
Records are lost when the program stops (kept in memory only)	Save to a text or CSV file using file handling
Fixed capacity of 100 students	Use a dynamic array that doubles in size, or a linked list
Search is a linear scan, $O(n)$	Keep the array sorted by roll number and use binary search, $O(\log n)$
Bubble sort is $O(n^2)$	Use merge sort or quick sort, $O(n \log n)$
Undo only covers deletion	Push every action onto the stack to undo edits as well
The server handles one request at a time	Handle each connection in a separate thread
No fee, attendance or room allocation records	Add a queue for the room waiting list — naturally FIFO

9. Questions a Teacher May Ask

Q1. Why did you use a stack here? Why not an array or a queue?

A. The stack stores deleted records for the undo feature. Undo must reverse the **most recent** action first, which is Last In First Out, and that is exactly what a stack provides. A queue would be wrong because it is First In First Out, so it would undo the *oldest* deletion first. A plain array would work only if I searched it for the newest entry every time, which is extra work the stack does automatically through `top_`.

Q2. Why did you not use `std::vector` or `std::stack`?

A. The purpose of the project is to demonstrate the data structures, so both are implemented by hand — the array with a `count_` variable and the stack with a `top_` index. Using the library containers would have hidden the exact operations, which are the point of the exercise.

Q3. What happens when you delete a student from the middle of the array?

A. The position is found by linear search, the record is pushed onto the stack, and then every element after that position is shifted one place to the left so no empty gap remains. Finally `count_` is decreased by one. In the worst case — deleting the first element — $n-1$ records must be shifted, so deletion is $O(n)$.

Q4. What are stack overflow and stack underflow, and how did you handle them?

A. Overflow is pushing onto a full stack; underflow is popping from an empty one. `push()` checks `top_ < MAX-1` before adding, and `undoDelete()` calls `stackEmpty()` (which tests `top_ == -1`) before popping. Both checks come **before** the array is touched, so the program never accesses memory outside the array.

Q5. Why does `top_` start at -1 instead of 0?

A. `top_` holds the index of the current top element. When the stack is empty there is no such element, so it must be an invalid index, and -1 is the value just before the first valid index 0. This also makes the size formula simple: the number of elements is always `top_ + 1`.

Q6. How does the search work if I type small letters but the data has capitals?

A. Both the search word and the stored text are first converted to small letters by `toLowerCase()`, which adds 32 to the ASCII value of any character between 'A' and 'Z' (because 'a' is 97 and 'A' is 65). The comparison is then done using `find()`, so `bca` matches `BCA`. The stored data itself is never changed — only the copies used for comparison.

Q7. How are two names compared for sorting?

A. `std::string` supports the `>` operator, which compares lexicographically: it compares the first characters, and if they are equal moves to the next, like a dictionary. The bubble sort in `sortByName()` uses this to swap records that are out of order, repeating until the whole array is sorted.

Q8. Which data structure would you add next, and why?

A. A **queue** for the room waiting list. Students should get rooms in the order they applied, which is First In First Out, so a queue fits naturally — just as undo fits a stack.

Q9. Is there JavaScript in the project?

A. No. The interface is only HTML and CSS. Every action is a normal HTML form or link, and the C++ program does all the processing and generates the page. This keeps all the logic in C++ where the data structures are.

Q10. Where is the data stored, and what happens when the program closes?

A. Everything is in memory, inside the array and the stack. When the program stops, the data is lost. Adding file handling to save and load records is the first planned improvement.

10. Conclusion

The project implements a working hostel student management system in which each data structure was chosen for a clear reason: an **array** for indexed storage and ordered display, a **stack** for the undo feature because undo is inherently LIFO, and **strings** with character level operations for case insensitive searching and alphabetical sorting. All three are implemented from first principles rather than taken from a library.

The same logic is exposed through two interfaces — a console menu and a browser page — which demonstrates that the data structure code is independent of how it is presented. Because all the logic sits in a single header file, `src/hostel.h`, it can be included directly in a larger system later.